

COURS DE BASES DE DONNEES I (LICENCE 2)

Enseignant:

Jeremie Kassouhuin

0197597272

I- Organiser les données selon la modélisation relationnelle

À quoi sert la modélisation relationnelle ?

Une base de données, à quoi ça sert ?

Comme son nom l'indique, une base de données (BDD), ça sert à stocker des données !

Mais une donnée, c'est quoi ? c'est très variable ! Cela peut être :

- une photo ;
- une vidéo ;
- du texte ;
- des nombres ;
- etc.

En général, ce ne sont pas des humains qui interagissent avec la base de données, mais plutôt des programmes informatiques (applis mobiles, logiciels, sites web, etc.). En fait, la base de données et le programme informatique (appli, site web, etc.)

sont **complémentaires** : l'un ne va pas sans l'autre. Si vous arrêtez un programme informatique qui n'est pas lié à une base de données, le programme ne se souviendra plus de rien.

C'est pour ça qu'il doit interagir avec la base de données: elle est en quelque sorte sa **mémoire**.

On peut comparer la BDD et le programme à un entrepôt et une usine : l'un ne fonctionne pas sans l'autre.

Un programme informatique, c'est un enchaînement de lignes de code qui effectuent des opérations : c'est comme les machines d'une usine. Ces machines, il faut les alimenter avec des matières premières, et ensuite, il faut stocker les produits finis : c'est le rôle de l'entrepôt. De la même manière, ce qui va alimenter les lignes de code du programme informatique, ce sont les données stockées dans la BDD.

Imaginez que l'entrepôt et l'usine appartiennent à une entreprise de fabrication de meubles. Dans l'entrepôt, on stocke les pièces détachées des meubles. Pour un lit par exemple, le sommier, le matelas et les 4 pieds seront tous stockés dans des parties différentes.

Un point qui aura son importance pour la suite : c'est une entreprise de luxe, et ce qui fait sa particularité, c'est que chaque meuble fabriqué est **unique**. En effet, chaque meuble est peint avec une couleur unique ! Certes, 2 lits peuvent être du même modèle et donc avoir la même forme, mais leurs sommiers seront forcément d'une couleur différente !

La modélisation relationnelle

Pour organiser un entrepôt, il faut bien connaître à l'avance ce qu'on va y stocker.

Pour une BDD, c'est le même principe : en fonction du type de données à stocker, on va choisir une structure appropriée. Dans ce cours, vous n'aborderez que le cas des données **structurées**.

Schématiquement, une donnée structurée, c'est une donnée que l'on peut facilement représenter sous forme de tableau (avec des lignes et des colonnes), comme vous l'avez sûrement déjà fait avec les logiciels LibreOffice Calc ou Excel. Les BDD faites pour accueillir des données structurées sont appelées **bases de données relationnelles**.

Vous l'aurez compris, il existe aussi des données moins structurées : on les rencontre souvent quand on a affaire à du Big Data. Pour cela, il faut des BDD non relationnelles. Elles ont des structures différentes, mais elles font quand même appel à des notions empruntées aux BDD relationnelles.

Conclusion : les données relationnelles, c'est LA base à connaître avant d'aller vous aventurer dans du Big Data. Et encore aujourd'hui, les bases relationnelles restent de loin les plus répandues, et les plus adaptées au fonctionnement basique des logiciels et des applis mobile ou web.

Définissez le concept de Table

L'objet que nous manipulerons tout au long de ce cours est appelé la **table**.

Une table, c'est simplement un tableau dans lequel on met des données, comme dans Excel ! Dans ce tableau, une ligne représente un objet, et chaque ligne représente des objets de même nature.

Reprenons l'exemple de l'entrepôt. Un entrepôt, c'est composé de différentes allées, toutes parallèles. Chaque allée est délimitée par une sorte de grande étagère où sont posées les pièces des meubles : on les appelle des rayonnages. Pour plus de simplicité, vous allez considérer que chaque rayonnage ne contient qu'un seul type de pièce : un rayonnage pour les matelas, un autre pour les sommiers, etc.

Eh bien dans une BDD relationnelle, chaque rayonnage correspond à une table. Il y a une table par type d'objet.

Identifiant matelas	Matière	Dimensions	Poids
M1	Latex	180x90x10	12 kg
M2	Latex	180x80x15	13 kg
M3	Coton bio	180x90x20	15 kg

Voici un exemple de table, qui liste des matelas. Chaque ligne correspond à un matelas, et chaque colonne donne une caractéristique du matelas en question, comme son poids, ses dimensions, etc.

Dans une table, 1 ligne = 1 objet, et toutes les lignes sont des objets de même nature.

Dans le jargon des bases de données, vous verrez qu'il existe plusieurs mots pour désigner la même chose. Certains de ces termes viennent de l'**algèbre relationnelle**, qui est la théorie qui sous-tend les bases de données relationnelles.

Par exemple, une table est parfois appelée une **relation** !

De même, le terme ***tuple*** désigne une ligne d'une table. Dans l'exemple précédent, le matelas M1 est représenté par le tuple suivant : (M1, Latex, 180x90x10, 12kg) . On emploie parfois aussi les mots *enregistrement* ou *vecteur*. Vous avez l'embarras du choix !

De la même manière, le terme ***attribut*** est l'équivalent d'une colonne. Dans l'exemple précédent, *matière* est un attribut.

Le **schéma d'une table** est l'ensemble des colonnes de cette table.

Rappel du vocabulaire:

Terme	Synonymes
table	relation
ligne	tuple, enregistrement, vecteur
colonne	attribut

Dans une table, l'ordre des lignes et l'ordre des colonnes n'ont pas d'importance, et donc pas de signification. Une table dans laquelle on mélangerait l'ordre des lignes, ou l'ordre des colonnes, représenterait exactement la même information.

En résumé

- Une base de données sert à stocker des données pour qu'elles soient accessibles par un ou plusieurs programmes/applications.
- La modélisation relationnelle permet de stocker des données facilement représentables sous forme de tableau.
- L'objet principal du modèle relationnel est la **table**, qui contient des **lignes** et des **colonnes**.
- Dans une table, 1 ligne = 1 objet.
- Dans une table, 1 colonne = 1 caractéristique.

Maintenant que vous savez ce qu'est une base de données, voyons dans le prochain chapitre comment la prendre en main et la manipuler.

II- Prenez en main une base de données

Initiez-vous au SGBDR

Concrètement, une BDD stockée sur un ordinateur, ça a quelle forme ?

Le plus souvent, une base de données est tout simplement composée d'**un ou plusieurs fichiers**, dans lesquels sont écrites toutes les données, un peu comme un fichier texte ou un fichier de tableur.

Certains programmes informatiques enregistrent leurs données dans leurs propres fichiers. Lorsque ces données sont peu nombreuses et peu complexes, alors cela fonctionne très bien. Cependant, dès que les données se complexifient, alors ce système n'est plus du tout optimal. Par exemple, lorsque les données doivent être partagées par plusieurs applications, il faut que ces applications se « mettent d'accord » sur la manière de **représenter** les données dans les fichiers. C'est couramment le cas dans les entreprises, où les données sont souvent **utilisées par plusieurs logiciels** : les données des ventes de produits sont par exemple utilisées par le logiciel du service Comptabilité. Ou encore les logiciels qui reportent, sous forme de graphiques, toutes les données clients utiles au service Marketing. Et si chaque logiciel lit et écrit des données de manière différente dans les mêmes fichiers, bonjour le bazar !

Aussi, **plus la quantité de données est importante, plus l'accès ou la modification de données est long**. En effet, il faut trouver dans de très gros (ou nombreux) fichiers, l'endroit exact où se trouve la donnée recherchée.

Pour faciliter cela, il existe beaucoup de **méthodes d'optimisation** permettant de réduire le temps d'accès à l'information.

Seulement voilà, utiliser toutes ces méthodes d'optimisation, c'est tout un art, et c'est complexe. Quand un développeur code un programme, il n'a pas le temps – voire pas les compétences – pour programmer toute cette optimisation.

C'est pour cela qu'ont été créés les **systèmes de gestion de bases de données (SGBD)**. Le SGBD, c'est un intermédiaire entre le programme et les données. Chaque SGBD utilise un langage afin de communiquer avec les applications qui souhaitent accéder aux données.

Si on reprend la métaphore de l'usine et de l'entrepôt, le SGBD, c'est le service qui gère l'entrepôt : il reçoit les commandes de l'usine, il lui envoie de qu'elle demande, et c'est lui qui gère l'optimisation du stockage : l'usine n'a pas à s'en soucier, elle laisse faire les pros !

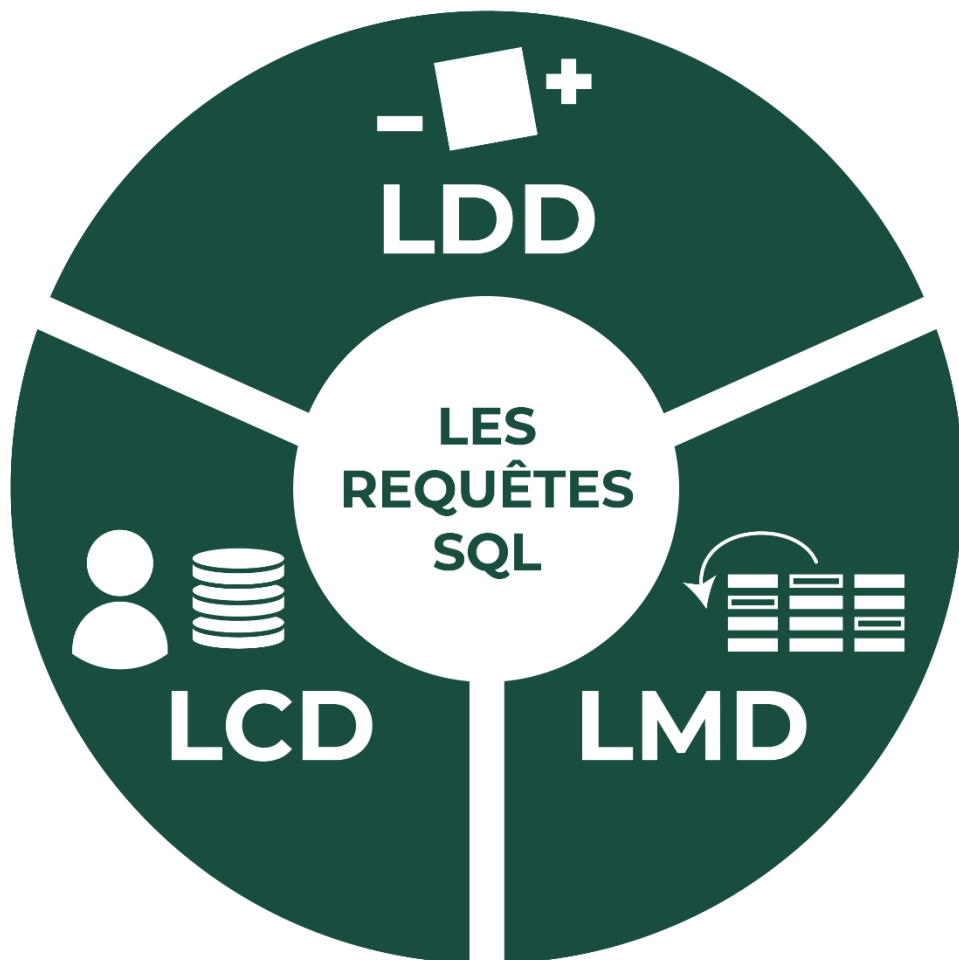
Les SGBD qui gèrent des bases de données relationnelles sont appelés **systèmes de gestion de bases de données relationnelles**, donc SGBDR. Les plus utilisés sont PostgreSQL, Oracle, MySQL, SQLite, etc. Le langage le plus utilisé par les SGBDR est le langage SQL (structured query language). C'est celui auquel vous allez vous former tout au long de ce cours !

Chaque SGBDR a sa propre manière d'interpréter le SQL. Heureusement, les fonctionnalités de base sont communes : le langage SQL que vous allez apprendre dans ce cours est applicable à tous les SGBDR. Mais il arrive que certains SGBDR ne prennent pas en charge certaines fonctionnalités plus avancées, ou bien que la syntaxe du SQL diffère légèrement d'un SGBDR à l'autre.

Parler en langage SQL, c'est formuler des **requêtes** à la base de données. Une requête peut avoir plusieurs buts : accéder à une donnée, ajouter une donnée, modifier la structure des tables, etc.

Les requêtes se répartissent en 3 groupes :

- le LDD (langage de définition de données) destiné à créer ou supprimer des objets dans la base de données (tables, contraintes, etc.) ;
- le LCD (langage de contrôle de données) qui gère les utilisateurs d'une base de données ainsi que leurs droits sur les objets (droit de consultation, de modification etc.) ;
- le LMD (langage de manipulation de données) destiné à manipuler les données contenues dans les tables, c'est-à-dire à manipuler les lignes de celles-ci. Les 4 opérations possibles sont :
 - la création de lignes,
 - la lecture de lignes,
 - l'actualisation de lignes,
 - la suppression de lignes.



Les 3 types de requêtes SQL

Dans ce cours, vous apprendrez le LMD, et plus particulièrement la partie **lecture** des données. Vous le verrez, toutes les requêtes de lecture de données commencent par le mot clé **SELECT**. Nous supposons donc que la base de données est déjà construite et déjà remplie. Ce qui nous intéresse ici, c'est d'aller fouiller dans les données pour y trouver ce qui nous intéresse. En effet, tout au long de ce cours, vous allez mener l'enquête !

Installez votre première base de données

Tout au long du cours, vous allez vous mettre dans la peau d'un data journaliste qui enquête sur le financement d'un réseau criminel.

Vous utiliserez pour votre enquête la base de données des Panama Papers, qui sont des données initialement secrètes, mais qui ont été révélées par vos collègues journalistes d'investigation.

Vous pouvez **télécharger cette base de données ici**

(<https://course.ocstatic.com/courses/7818671/panamapapers.zip>), elle est au format SQLite 3.

Vous pouvez l'ouvrir grâce à tout logiciel qui prend en compte ce format, mais je vous conseille SQLiteStudio, que vous pouvez **télécharger à ce lien** <https://sqlitestudio.pl/>.

Découvrez l'architecture de votre base de données

Vous avez, au cours de votre enquête, intercepté une facture émise par une mystérieuse société (c'est-à-dire une entreprise) qui s'appelle « Big Data Crunchers Limited ». Sur ces factures, l'adresse de la société n'est pas indiquée.

Vous ne savez pas qui se cache derrière cette mystérieuse entreprise, mais vous pensez qu'il peut s'agir d'une société-écran.

Une **société-écran**, c'est une société fictive créée pour dissimuler les transactions financières d'une ou de plusieurs autres sociétés. Elles sont généralement organisées de manière à ce que la personne qui touche réellement les bénéfices générés par la société ne soit pas ou difficilement identifiable.

Dans votre base de données, la liste des sociétés (dont certaines sont des sociétés-écrans) se trouve dans la table `entity` , (ce qui signifie « entité », en français).

Souvent, les sociétés-écrans sont domiciliées dans des paradis fiscaux, qui sont des pays (ou plus précisément des juridictions) souvent caractérisés par une forte opacité sur les données bancaires, et/ou par des taux de taxation sur les sociétés très bas.

La personne à qui reviennent réellement les bénéfices d'une société s'appelle le ou la **bénéficiaire**.

Votre base de données contient un bon nombre de ces bénéficiaires, qui sont contenus dans la table `officer` .

Mais une société ne se crée pas si facilement que ça. En général, il faut demander de l'aide à des services spécialisés. On les appelle souvent des intermédiaires, qui sont principalement des cabinets d'avocats, des banques ou des personnes. Ils se trouvent dans la table `intermediary` .

Vous allez donc enquêter sur la mystérieuse société Big Data Crunchers Ltd, mais également sur les intermédiaires qui ont aidé à la créer. Afin de rédiger votre article, vous regarderez si ces intermédiaires ont une grosse activité dans des paradis fiscaux, ce qui pourrait vous mener sur la piste d'un réseau de blanchiment d'argent.

Créez votre première table

```
CREATE TABLE entity2 (  
    id INTEGER,  
    name TEXT NOT NULL,  
    jurisdiction TEXT,  
    jurisdiction_description TEXT,  
    incorporation_date DATE,  
    status TEXT,  
    service_provider TEXT,  
    id_address INTEGER,  
    source TEXT,  
    note TEXT,  
    end_date DATE,  
    url TEXT,  
    lifetime INTEGER,  
    PRIMARY KEY(id)  
)
```

Ajout de données dans la table entity2 :

```
INSERT INTO  
    entity2 (id, name, jurisdiction, jurisdiction_description, incorporation_date)  
VALUES  
    (1, 'Une société', 'IMG', 'Le Pays Imaginaire', '2024-01-01'),  
    (2, 'Une autre société', 'IMG', 'Le Pays Imaginaire', '2025-01-01'),  
    (3, 'Encore une société', 'IMG', 'Le Pays Imaginaire', '2026-01-01');
```

C'était juste pour tester ; pour ne pas vous embrouiller par la suite, supprimez la table en tapant `DROP TABLE entity2`.

À vous de jouer



Contexte

La personne qui vous a fourni la fameuse facture émise par la société *Big Data Crunchers Limited* (on appellera cette personne votre “source” ; elle souhaite rester anonyme), travaille dans une entreprise. Mais cette entreprise est bien réelle : elle a de vrais bâtiments, des salariés en chair et en os ; bref, ce n'est pas une société-écran. Cette entreprise est mentionnée comme cliente de la facture sur laquelle vous enquêtez. En plus de cette facture, votre source vous a aussi secrètement fourni la liste des employés sous forme d'une table, que nous appellerons ici la table `personne` .

Consignes

Voici sur cette illustration la table `personne` que vous souhaitez stocker dans votre base.

nom	prenom	telephone_professionnel	numero_bureau	identifiant	departement
Dom	Malika	01 29 38 ** 01	27	2893	ressources humaines
Dirichlet	John	01 29 38 ** 02	01	2983	marketing
Hati	Hassia	01 29 38 ** 03	12	1829	marketing
Bernard	George	Null	51	2993	maintenant

Créez la requête SQL qui créera la *structure* de cette table (juste la structure, pas besoin d'y insérer les données).

Indices :

- dans une table, l'ordre des colonnes n'a pas d'importance ;
- la clé primaire de la table s'écrit ici PRIMARY KEY(identifiant) ;
- le numéro de téléphone est considéré comme du texte, pas comme un numéro.

Vérifiez votre travail

Il y a plusieurs solutions possibles. Je vous en donne une un peu plus bas. Mais pour vérifier si votre réponse est bonne, il vous suffit d'exécuter ces 2 requêtes (l'une après l'autre). Si la 2nde requête affiche les mêmes données que l'illustration de la table donnée ci-dessus, alors vous avez bon !

```
INSERT INTO personne (nom, prenom, telephone_professionnel, numero_bureau, identifiant,
departement)
```

```
VALUES
```

```
('Dom','Malika','01 29 38 ** 01', 27, 2893,'ressources humaines'),
('Dirichlet','John','01 29 38 ** 02',01,2983,'marketing'),
('Hati','Hassia','01 29 38 ** 03',12,1829,'marketing'),
('Bernard','George', NULL, 51, 2993, 'maintenance')
```

```
SELECT * FROM personne
```

Voici une réponse possible :

```
CREATE TABLE personne (
    nom TEXT,
    prenom TEXT,
    telephone_professionnel TEXT,
    numero_bureau INTEGER,
    identifiant INTEGER,
    departement TEXT,
    PRIMARY KEY(identifiant)
)
```

Si vous n'avez plus besoin de cette table, supprimez-la en tapant DROP TABLE personne .

En résumé

- Un SGBD est un programme qui fait l'**intermédiaire** entre le programme / l'application et les données, à l'aide d'un langage prédéfini, comme le SQL, par exemple.

- Le SGBD est spécialisé dans le **stockage** de données, et **optimise** les accès en lecture et en écriture.
- On communique avec le SGBDR avec des **requêtes** SQL.
- On crée une table avec `CREATE TABLE` .
- On insère des données grâce à `INSERT INTO (...) VALUES (...)` .

Vous en savez un peu plus sur l'utilisation du SGBD pour naviguer dans votre base de données. Dans le prochain chapitre, nous verrons comment utiliser la clé primaire pour identifier les lignes de votre table.

III- Identifiez les lignes de votre table avec votre clé primaire

Qu'est-ce qu'une clé primaire ?

C'est quoi une clé ?

Dans la vie quotidienne, une clé sert à ouvrir une porte pour **accéder** à votre maison. Une caractéristique essentielle d'une clé est que sa forme doit être **unique**, car si votre clé est identique à celle de la maison de votre voisin, il se pose un problème évident ! Comme il n'y a qu'une clé par maison, on peut dire que votre clé, c'est un peu la « carte **d'identité** » de votre maison !

On vient de voir ici deux notions fondamentales : celle de **l'identité** et celle de **l'unicité**. Et comme nous venons de le voir, ces notions sont intimement liées !

Dans une table, une clé a donc pour vocation d'accéder à une ligne, et donc de l'identifier. Cette clé, on l'appelle **clé primaire**.

Dans la plupart des SGBDR, vous serez obligé e de donner une clé à chacune de vos tables.

Pour déterminer une clé primaire pour votre table, il faut vous poser la question suivante :

Quelles sont les informations minimales dont j'ai besoin pour identifier de manière unique un objet ?

Cette question peut être reformulée en « Quelles sont les colonnes dont j'ai besoin au minimum pour identifier à coup sûr une ligne de ma table ? ». Ce que cela signifie, c'est que si on est sûr e que n'importe quelle ligne peut être retrouvée en connaissant la valeur de quelques-unes de ses colonnes, c'est que ce groupe de colonnes peut être une clé primaire !

Cette illustration vous aidera peut-être à visualiser ce concept :

CLÉ PRIMAIRE		nom	prénom	bureau	département
attribut 1	attribut 2				
		Dom	Malika	27	ressources humaines
		Dirichlet	John	01	marketing
		Hati	Hassia	12	marketing
		Bernard	Georges	51	maintenance

Une clé primaire est déterminée par des attributs

Je vous donne un exemple juste en dessous.

Définissez la clé primaire adaptée

La notion **d'identification** est essentielle pour les facteurs qui doivent identifier de manière certaine les logements pour lesquels ils distribuent le courrier. Imaginez une table qui répertorie des logements, possédant ces colonnes :

1. Type de logement (maison, appartement, etc.).
2. Superficie.
3. N° de logement (ex. : appartement numéro 203).
4. N° de voie (ex. : 5).
5. Complément (ex. : 5 bis).
6. Intitulé de la voie (ex. : rue de la République).
7. Commune (ex. : Passillac-les-Flots).
8. Code postal.
9. Pays.

La clé primaire de cette table, ce sera le groupe de colonnes minimum que l'on peut donner au facteur pour qu'il soit sûr de pouvoir distribuer un courrier au bon logement.

Si on lui donne uniquement les colonnes intitulé de la voie , code postal et commune (par exemple « rue de la République, 93124 Passillac-les-Flots »), il lui sera impossible de donner le courrier, car la rue de la République de Passillac contient plusieurs logements.

Si au contraire, on lui donne toutes les colonnes, le courrier pourra à coup sûr être distribué.

Cependant, le groupe de colonnes 1 à 9 n'est pas considéré comme minimal : la superficie n'a pas besoin d'être connue pour distribuer un courrier. superficie ne doit donc pas figurer dans la clé primaire.

Une bonne clé primaire pour cette table sera donc le groupe des colonnes 3 à 9 (remarque : même si pour une maison, la colonne n° de logement n'a pas besoin d'être connue, il est quand même important de la placer dans la clé primaire de la table, afin de pouvoir identifier les différents appartements d'un même immeuble).

Il arrive parfois que pour une table, il y ait plusieurs clés primaires possibles. Dans ce cas, on en choisit une (de préférence la plus simple).

Les colonnes qui composent la clé primaire d'une table sont souvent indiquées par les initiales anglaises **PK**, pour « primary key » :

immatriculation [PK]	pays_immatriculation [PK]	couleur	nombre_p
2983-AA	France	bleue	5
1923-BD	France	verte	7

Exemple de Primary Key abrégée PK dans une base de données

Allez, maintenant que vous savez ce qu'est une clé primaire, je vous donne la définition “officielle” :

La clé primaire d'une table est un groupe de colonnes minimum permettant d'identifier de manière unique une ligne d'une table.

Voici une formulation alternative plus détaillée : “Pour qu'un groupe de colonnes soit une clé, il faut être sûr que deux lignes n'auront jamais des valeurs identiques pour ces colonnes. Ce groupe de colonnes doit être minimal, c'est-à-dire que si on retire l'une des colonnes de ce groupe, la phrase précédente n'est plus vérifiée.”

La contrainte d'unicité

Dans l'exemple ci-dessus, en choisissant les colonnes 3 à 9 comme clé primaire de la table `Logement`, on définit implicitement que deux lignes de cette table ne peuvent pas avoir des valeurs identiques pour ces colonnes. Le SGBDR vous l'interdira.

On dit que la clé primaire définit une **contrainte d'unicité** sur le groupe de colonnes 3 à 9.

Autrement dit, le groupe `(code_postal, commune)` ne peut pas être une clé primaire car les lignes « 3 rue de la Charrue-avant-les-boeufs, 93124 Passillac-les-Flots » et « 5 rue du Pangolin-superstitieux, 93124 Passillac-les-Flots » ont toutes les deux les *mêmes valeurs* pour `(code_postal, commune)` : ces valeurs sont 93124 et Passillac-les-Flots. Ce groupe de colonnes violerait la contrainte d'unicité.

Une clé primaire définit la nature des lignes de la table

Quand vous êtes en présence d'une table, il est très important de connaître la clé primaire, car c'est ce qui *détermine la nature des objets* qu'il y a dans la table. Je m'explique.

Si on reprend l'exemple du courrier à acheminer. Imaginons 2 cas :

1. La clé primaire est `code_postal`.
2. La clé primaire est le groupe de colonnes 3 à 9.

Si on se place dans le cas 1, cela signifie que 2 lignes de la table ne peuvent pas avoir le même code postal. Il en découle qu'il n'y a qu'une ligne par code postal. Et s'il n'y a qu'une ligne par code postal, ça veut dire qu'une ligne représente un code postal. Autrement dit, « 1 ligne = 1 code postal ».

Si par contre on se place dans le cas 2, alors cela signifie qu'il n'y a pas 2 lignes de la table qui ont la même adresse. Donc « 1 ligne = 1 adresse ».

Vous voyez, la clé primaire vous aide à savoir quelle est la nature des lignes de votre table !

En pratique, on crée souvent des clés artificielles (des identifiants) dans les tables, même quand on pourrait former une clé primaire à partir des colonnes déjà existantes. C'est tout à fait OK, mais il ne faut pas que cela vous dispense de vous poser la question « Quelle est la nature des lignes de cette table ? ». Sinon, vous pouvez faire de graves erreurs : vous verrez cela dans l'un des chapitres qui vont suivre.

Clé artificielle

Parmi les colonnes d'une table, il est parfois impossible de trouver une clé primaire. Dans d'autres cas, la clé primaire est trop complexe (trop de colonnes, par exemple). Dans ce cas, il est possible de créer une **clé primaire artificielle**.

Une clé artificielle est une colonne que l'on ajoute à la table. Cette colonne n'a pas de réelle signification (contrairement aux autres colonnes), mais sa seule fonction est d'identifier de manière unique les lignes de la table. C'est donc un **identifiant**.

Le numéro de Sécurité sociale et le numéro de carte d'identité sont des clés artificielles permettant d'identifier les personnes de nationalité française.

Représentez votre clé primaire

En SQL, on définit la clé primaire grâce au mot clé PRIMARY KEY, et on écrit ensuite la/les colonne(s) qui constituent la clé primaire.

C'est ce que vous aviez fait pour votre table `entity2` , rappelez-vous :

```
CREATE TABLE entity2 (  
    id INTEGER,  
    name TEXT NOT NULL,  
    jurisdiction TEXT,  
    jurisdiction_description TEXT,  
    incorporation_date DATE,  
    status TEXT,  
    service_provider TEXT,  
    id_address INTEGER,  
    source TEXT,  
    note TEXT,  
    end_date DATE,  
    url TEXT,  
    lifetime INTEGER,  
    PRIMARY KEY(id)  
)
```

Ici, la clé primaire était une clé artificielle, soit la colonne `id` .

Mais rappelez-vous qu'une clé primaire peut être composée de plusieurs colonnes. C'était le cas avec la table des adresses plus haut dans ce chapitre. Pour cette table, on aurait écrit ceci :

```
CREATE TABLE adresse_postale (
  type_logement TEXT,
  superficie INTEGER,
  numero_logement INTEGER,
  numero_voie INTEGER,
  complement TEXT,
  intitule_voie TEXT,
  commune TEXT,
  code_postal TEXT,
  pays TEXT,
  PRIMARY KEY (numero_logement, numero_voie, complement, intitule_voie, commune,
code_postal, pays)
)
```

À vous de jouer



Contexte

Ayant appris que vous travaillez sur les Panama Papers, l'un de vos collègues journalistes d'investigation vous envoie un fichier Excel issu d'une table d'une base de données relationnelle. Voici son message :

Tiens, l'une de mes sources m'a transmis ceci. Cette personne (ma source) a eu accès aux comptes mails de 2 cabinets d'avocats ayant des activités au Panama, dont voici les noms : *Avocats Conseils Corp.* et *Guacamole Associates Ltd.*

Malheureusement, elle n'a pas pu récupérer les e-mails de ces comptes, mais elle a pu récupérer le carnet d'adresses de chacun de ces 2 comptes mails. C'est mieux que rien : ces carnets d'adresses contiennent des contacts (nom, prénom, e-mail, date de création du contact).

Même s'il n'y a pas de preuve formelle, il est très probable qu'une personne (un contact) présente dans l'un des 2 carnets d'adresses ait eu des échanges avec l'un des 2 cabinets d'avocats ! Tu me parlais notamment d'une personne sur laquelle tu enquêtais, Annie Versaire : son adresse mail était enregistrée dans le carnet d'adresses de *Avocats Conseils Corp.* ; cela peut t'orienter dans ton enquête !

Consignes

Vous décidez de rentrer cette table dans votre SGBDR. Choisissez une clé primaire, puis écrivez le code SQL qui permet de créer cette table (juste la table, pas les données qui vont dedans):

compte_mail	mail	nom	prenom	date_creation
Avocats Conseils Corp.	eva.poree.389@gmx.com	Eva	Porée	2010-01-03
Avocats Conseils Corp.	ali.gator2892@protonmail.com	Ali	Gator	2010-02-01
Avocats Conseils Corp.	annie_versaire2983@gmx.com	Annie	Versaire	2010-10-04
Guacamole Associates Ltd.	alex_ception@imag.com	Alex	Ception	2010-02-02
Guacamole Associates Ltd.	ali.gator2892@protonmail.com	Ali	Gator	2009-09-10
Guacamole Associates Ltd.	eva.poree.389@gmx.com	Eva	Poree	2010-05-13

Vérifiez votre travail

```
CREATE TABLE compte (
  compte_mail TEXT,
  mail TEXT,
  nom TEXT,
  prenom TEXT,
  date_creation DATE,
  PRIMARY KEY (compte_mail, mail)
)
```

Le clé primaire est (compte_mail, mail) .

En effet, l'adresse mail du contact ne peut pas suffire à lui seul, car 2 adresses mail peuvent être présentes dans 2 carnets d'adresses différents (c'est d'ailleurs le cas des adresses mail d'Ali Gator

et d'Éva Porée). Les noms et prénoms ne peuvent pas être inclus dans la clé primaire, car il est possible que plusieurs personnes aient à la fois le même prénom et le même nom.

En résumé

- La clé primaire d'une table est un groupe de colonnes minimum permettant d'identifier une ligne d'une table.
- Une clé primaire a donc forcément une contrainte d'unicité.
- Dans certains cas, il est impossible de trouver une clé primaire parmi les colonnes présentes. Dans d'autres cas, la clé primaire est trop complexe. On crée alors une clé primaire artificielle....
- Une clé primaire se dit en anglais *primary key* (PK).

Vous en savez plus sur les clés primaires désormais. Dans le prochain chapitre, nous verrons comment mettre en relation plusieurs tables à l'aide de clés étrangères.

IV- METTRE EN RELATION PLUSIEURS TABLES AVEC DES CLES ETRANGERES

Qu'est-ce qu'une clé étrangère ?

Au chapitre précédent, vous avez vu comment identifier les lignes de vos tables.

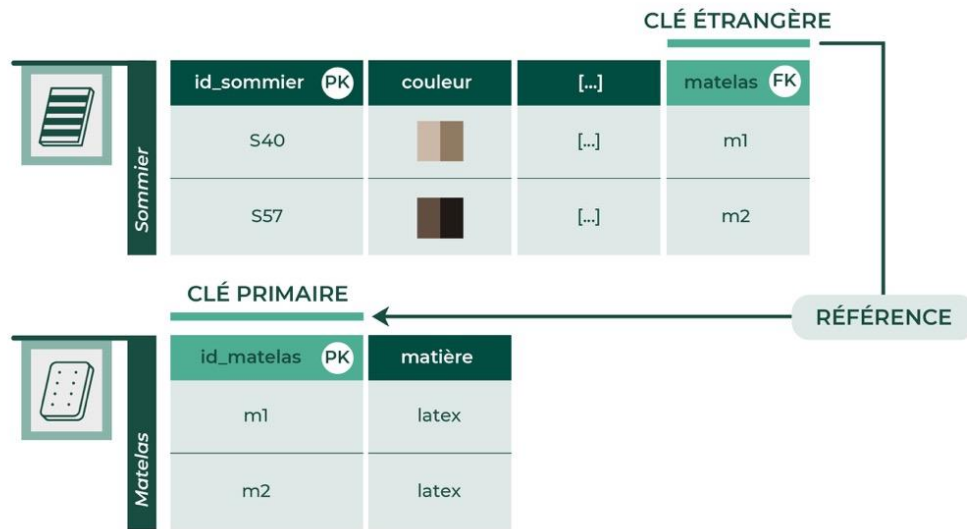
Mais votre base ne contient pas qu'une table, et il arrive que 2 tables contiennent des informations qui sont liées entre elles.

Par exemple, votre table `sommier` est en lien avec la table `matelas`, car chaque sommier est associé au matelas avec lequel il sera assemblé !

Pour lier des tables entre elles, on a créé le concept de **clé étrangère**.

Une clé étrangère, c'est une colonne (ou groupe de colonnes) d'une table qui fait référence à la clé primaire d'une autre table, afin de modéliser le lien entre les lignes de ces deux tables.

Voici ce que cela donne pour les tables `sommier` et `matelas` :



Une clé étrangère sert à lier deux tables ensemble

Les colonnes qui constituent une clé étrangère sont désignées par le signe « FK », pour « Foreign Key ».

Vous voyez qu'il y a une colonne dans **sommier** qui contient des identifiants de **matelas**. C'est comme cela que l'on sait que le sommier S40 va être assemblé avec le matelas M1.

Les valeurs présentes dans la colonne **matelas_fk** doivent obligatoirement avoir une correspondance dans la colonne **id_matelas**, sinon les données sont incohérentes. La plupart des SGBDR n'accepteront pas que vous introduisiez un sommier dont le matelas indiqué dans la colonne **matelas_fk** n'existe pas dans la table **matelas**.

Ici, la clé étrangère n'est composée que d'une colonne (**matelas_fk**), car la clé primaire de la table **matelas** ne contient qu'une colonne (**id_matelas**). Mais si la clé primaire contient deux colonnes, alors il faut une clé étrangère à deux colonnes !

À vous de jouer



Contexte

Vous commencez à vous familiariser avec votre base de données des Panama Papers. Vous y voyez 2 tables qui semblent être liées : la table `entity` et la table `address`. Si elles sont liées, il y a probablement une clé étrangère quelque part. La repérer vous aidera ensuite à chercher plus efficacement et de manière plus cohérente dans vos données.

Consignes

Repérez la clé étrangère, déterminez de combien de colonnes elle est composée, et trouvez la clé primaire qu'elle référence :

Table `entity` :

id	name	jurisdiction	jurisdiction_description	incorporation_date	status	id_address
10005216	SHINY FASHION CO., LTD.	SAM	Samoa	2015-12-18	Active	132816168698714
10003078	NEWTOP ENTERPRISE LIMITED	SAM	Samoa	2015-12-17	Active	132816168698714
10166106	FUNDACION JUDARALI	PMA	Panama	2015-12-17	Active	32462814263035
10191495	TONY TRADING LIMITED	SEY	Seychelles	2015-12-16	Active	17546640025346
10165314	STILES FOUNDATION	PMA	Panama	2015-12-15	Active	134544105071872

La table `entity`

Table `address` :

id_address	address	country	country_code
17546640025346	OFFSHORER BUSINESS SERVICES (HK) LTD. ...	Hong Kong	HKG
32462814263035	CONTEXTO LEGAL CALLE 11B, NO. 41 - 53 ...	Colombia	COL
35048805670689	MOSSACK FONSECA & CO. CZECH INC. LEVEL ...	Samoa	WSM
36435271214138	MOSSACK FONSECA & CO. MOSSFON BUILDI...	Panama	PAN
132816168698714	HANGZHOU LINGC COMMERCIAL LTD. ROOM...	China	CHN
134544105071872	ARTURO ROBERTO QUEVEDO WIDMANN C/...	Guatemala	GTM

La table `address`

Vérifiez votre travail

La clé étrangère est composée d'une seule colonne : `id_address` de la table `entity` . Elle référence la clé primaire de `address` , qui est la colonne `id_address` . En effet, les valeurs dans la colonne `entity.id_address` semblent être des identifiants. Ces identifiants se retrouvent dans la colonne `address.id_address` . Vu le nom de cette colonne ("`id_address`"), il est très probable qu'elle soit la clé primaire artificielle de la table `address`.

Créez du lien entre vos tables

Maintenant que vous savez reconnaître une clé étrangère, on va voir comment implémenter ce concept en SQL.

Vous avez déjà vu le code permettant de créer la table `entity`. Mais la clé étrangère n'était pas indiquée. Pour spécifier la clé étrangère entre `entity` et `address` , il faut ajouter ce bout de code lors de la création de la table `entity` :

`FOREIGN KEY ( <colonnes clés étrangères> ) REFERENCES <table référencée> ( <clé primaire de la table référencée> )`

Ce qui donne :

```
CREATE TABLE entity2 (  
    id INTEGER,  
    name TEXT NOT NULL,  
    jurisdiction TEXT,  
    jurisdiction_description TEXT,  
    incorporation_date DATE,  
    status TEXT,  
    service_provider TEXT,  
    id_address INTEGER,  
    source TEXT,  
    note TEXT,  
    end_date DATE,  
    url TEXT,  
    lifetime INTEGER,  
    PRIMARY KEY(id),  
    FOREIGN KEY (id_address) REFERENCES address (id_address)
```

)

Cette requête ne fonctionnera que si la table `address` existe déjà dans la base (c'est le cas si vous exécutez cette requête dans la base des Panama Papers). Si ce n'est pas le cas, il faudra la créer d'abord.

Supprimez ensuite `entity2` avec `DROP TABLE entity2` .

En résumé

- Une **clé étrangère** est un groupe de colonnes d'une table, qui fait référence à la clé primaire d'une autre table, afin de modéliser le lien entre les lignes de ces deux tables.
- Une clé étrangère peut être composée d'**une ou plusieurs colonnes**.
- Une clé étrangère se dit en anglais *foreign key* (FK).
- En SQL, une clé étrangère peut se définir lors de la création de la table, en écrivant `FOREIGN KEY ( <colonnes clés étrangères> ) REFERENCES <table référencée> ( <clé primaire de la table référencée> )`

Maintenant que vous savez utiliser les clés étrangères, nous allons voir dans le prochain chapitre comment identifier les différents types d'associations entre vos tables.

V- IDENTIFIER LES TYPES D'ASSOCIATIONS ENTRE LES TABLES

À ce stade, vous savez qu'une clé étrangère sert à créer une association entre une table A et une table B. Voyons maintenant les 3 cas de figure dans lesquels on les utilise :

- l'association un-à-un ;
- l'association un-à-plusieurs ;
- l'association plusieurs-à-plusieurs.

En fait, tout est question du nombre de lignes de A qui peuvent être associées aux lignes de B, et inversement !

Par exemple, prenons l'association entre les sommiers et les matelas. D'un côté, un sommier est associé à **un** matelas. De l'autre côté, un matelas est associé à **un** sommier. On est donc face à une association **un-à-un**.

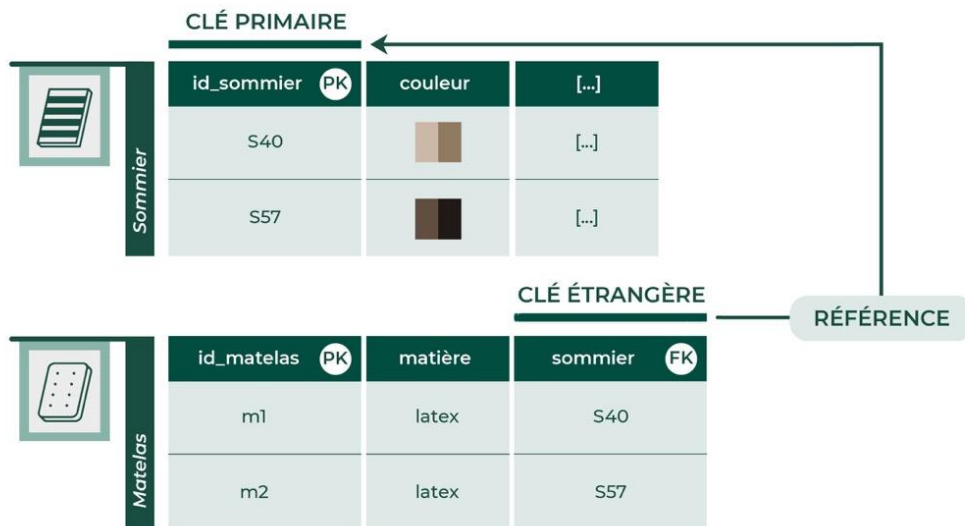
L'association **un-à-plusieurs** se rencontre par exemple entre les sommiers et les pieds du lit : un sommier est associé à **plusieurs** pieds, et un pied est associé à **un** sommier.

Imaginons maintenant qu'un nouveau catalogue de literie soit édité tous les 2 ans. Un sommier peut être présent dans **plusieurs** catalogues, et un catalogue présente **plusieurs** sommiers. On est ici dans le cas **plusieurs-à-plusieurs**.

Implémentez une association un-à-un

Au chapitre précédent, vous avez lié les tables `matelas` et `sommier` en ajoutant une clé étrangère dans `sommier`.

Mais vous auriez aussi pu faire l'inverse : mettre la clé étrangère dans `matelas`, comme ceci :



Exemple d'association un-à-un

Ici, on a cette liberté car il y a un matelas par sommier, et un sommier par matelas. On met donc la clé étrangère soit dans la table **sommier**, soit dans la table **matelas**. Nous sommes dans le cas d'une association **un-à-un**.

Implémentez une association un-à-plusieurs

Reprenons l'exemple du chapitre précédent : celui de l'association entre les tables `entity` et `address`. Ici, nous sommes dans un cas où une société ne peut avoir qu'une seule adresse maximum, mais une adresse peut accueillir plusieurs sociétés.

Nous sommes dans le cas d'une association **un-à-plusieurs**.

Dans ce cas, on utilise aussi une clé étrangère, comme vous l'avez vu précédemment :

Table `entity` :

id	name	jurisdiction	jurisdiction_description	incorporation_date	status	id_address
10005216	SHINY FASHION CO., LTD.	SAM	Samoa	2015-12-18	Active	132816168698714
10003078	NEWTOP ENTERPRISE LIMITED	SAM	Samoa	2015-12-17	Active	132816168698714
10166106	FUNDACION JUDARALI	PMA	Panama	2015-12-17	Active	32462814263035
10191495	TONY TRADING LIMITED	SEY	Seychelles	2015-12-16	Active	17546640025346
10165314	STILES FOUNDATION	PMA	Panama	2015-12-15	Active	134544105071872

La table `entity`

Table `address` :

id_address	address	country	country_code
17546640025346	OFFSHORER BUSINESS SERVICES (HK) LTD. ...	Hong Kong	HKG
32462814263035	CONTEXTO LEGAL CALLE 11B, NO. 41 - 53 ...	Colombia	COL
35048805670689	MOSSACK FONSECA & CO. CZECH INC. LEVEL ...	Samoa	WSM
36435271214138	MOSSACK FONSECA & CO. MOSSFON BUILDI...	Panama	PAN
132816168698714	HANGZHOU LINGC COMMERCIAL LTD. ROOM...	China	CHN
134544105071872	ARTURO ROBERTO QUEVEDO WIDMANN C/...	Guatemala	GTM

La table `address`

Vous voyez que les 2 premières lignes de `entity` ont la même adresse (l'adresse numéro 132816168698714).

Ici, on est un peu plus contraint que pour l'association un-à-un. En effet, vous n'avez pas le choix de la table dans laquelle placer la clé étrangère. Il faut la placer dans la table qui est du côté "plusieurs" (ici `entity`). J'ai donné la raison dans la vidéo en haut de ce chapitre.

Implémentez une association plusieurs-à-plusieurs

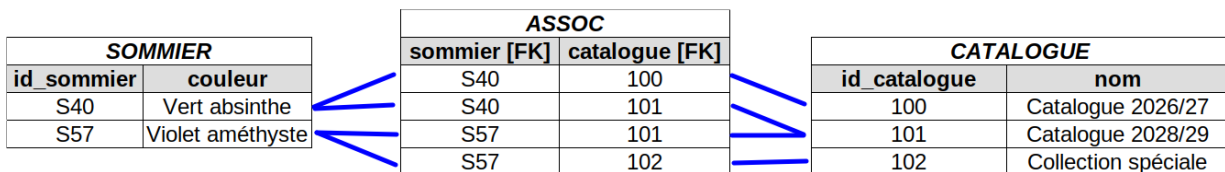
Passons maintenant au cas de l'association **plusieurs-à-plusieurs**. C'est quand une ligne d'une première table peut être associée à plusieurs lignes d'une seconde table, et qu'une ligne de cette seconde table peut être associée à plusieurs lignes de la première table.

Prenez un temps de réflexion avec l'exemple des sommiers et des catalogues : un catalogue contient plusieurs sommiers, et un sommier peut être présent dans plusieurs catalogues.

Essayez de résoudre ce problème en plaçant une clé primaire dans l'une ou l'autre table. Dans tous les cas, vous allez vous retrouver bloqué, car si vous placez la clé étrangère dans **sommier**, alors un sommier ne sera associé qu'à un seul catalogue ! Et si vous placez la clé étrangère dans catalogue, un catalogue ne pourra contenir qu'un seul sommier...

La solution, c'est de déplacer le problème vers... une troisième table !

On l'appelle **table d'association**.



Exemple de table association

Comme vous le voyez, elle contient 2 clés étrangères, pointant chacune vers l'une des 2 tables d'origine. Ainsi, cette ligne de la table 1 peut être associée à plusieurs lignes de la table 2, et vice versa !

La clé primaire de cette table d'association est composée *au moins* des 2 clés étrangères.

Pourquoi au moins ?

Car il est possible de donner des caractéristiques supplémentaires à la table d'association, comme ici :

sommier [FK]	catalogue [FK]	pays	prix
S40	100	France	180 €
S40	100	USA	200 \$
S40	101	France	170 €
S40	101	USA	180 \$
S57	101	France	250 €

Ici, on a ajouté des caractéristiques aux associations entre sommiers et catalogues. On y voit que le sommier S40, dans le catalogue 2026-2027, est vendu 180 € en France, ou 200 \$ aux États-Unis. Dans le catalogue 2028-2029, les prix ont baissé.

Ici, la clé primaire de cette table est composée de 3 colonnes : [sommier, catalogue, pays].

Dans la base de données des Panama Papers, nous avons un exemple de table d'association ayant une clé primaire avec une 3e colonne : c'est la table `assoc_officer_entity` . Elle représente l'association plusieurs-à-plusieurs entre la table `entity` et la table `officer` . Sa clé primaire est (officer, entity, assoc_type). Cette dernière colonne donne le type d'association qui lie une personne présente dans `officer` et une société de `entity` . Par exemple, elle peut donner le président d'une société, ainsi que ses actionnaires. Ici, le groupe de colonnes (officer, entity) ne peut pas être à lui seul une clé primaire, car une même personne peut être à la fois présidente et actionnaire d'une société :

officer [PK]	entity [PK]	assoc_type [PK]	start_date	end_date
M. Pédégé	Doumatelas Corp.	président	01-01-2000	04-03-2014
M. Pédégé	Doumatelas Corp.	actionnaire	01-04-2000	null
Mme Bigboss	Bonne nuit Ltd.	présidente	01-01-2002	12-03-2010

Si vous avez appris à modéliser une base de données en utilisant le langage UML, vous savez que le fait de donner des caractéristiques à une association est équivalent à écrire une **composition**. D'ailleurs, vous pouvez très bien appeler la table ci-dessus **table de composition** au lieu de table d'association, c'est à vous de choisir.

En résumé

- Pour une association **un-à-un** entre une table `A` et une table `B` , on utilise une clé étrangère. La clé étrangère est placée dans `A` et elle référence la clé primaire de `B` , ou bien l'inverse.
- Une association **un-à-plusieurs** se traduit en ajoutant une clé étrangère dans la table qui est du côté « plusieurs ». Cette clé étrangère référence la clé primaire de la table qui est du côté « un ».

- Une association **plusieurs-à-plusieurs** se traduit en ajoutant une nouvelle table. La clé primaire de cette nouvelle table sera composée (au moins) des deux clés étrangères référençant les deux tables à associer.

Vous voici parvenu au bout de cette première partie. Je vous propose de tester vos connaissances dans le quiz suivant avant de vous attaquer à la seconde partie du cours.